1. Introduction of python :

Q:1) Introduction to Python and its Features (simple, high-level, interpreted language).

Ans: Python is one of the most popular programming languages in the world. It was created by Guido van Rossum and first released in 1991.

- Simple language:
  Python has an easy and readable syntax similar to English. No matter are you beginner or higher level developer this language is suitable for all.

- High-Level Language:
  Python allows programmers to focus on solving problems without worrying about hardware or memory management details.

- Interpreted Language:
  Python executes code line by line, making debugging and testing easier.

Q:2) History and evolution of Python.

Ans: Python is a popular programming language that was created by Guido van Rossum in 1991. He developed Python with the goal of making programming simple, easy to read, and easy to learn. The name "Python" was inspired by the television show Monty Python's Flying Circus.

The first version of Python, **Python 1.0**, was released in 1994. As Python became more popular, new versions were released with improved features and better performance. In 2000, **Python 2.0** was introduced with advanced features like garbage collection and support for Unicode. Later, in 2008, **Python 3.0** was released. It fixed many design issues from previous versions and made the language more efficient and user-friendly. Today, Python 3 is the standard version used worldwide.

Over the years, Python has evolved into one of the most widely used programming languages. In result Python has grown from a simple programming language into a powerful tool used across many industries.

Q:3) Advantages of using Python over other programming languages.

Ans: **1. Easy to Learn:**

Python has simple syntax, so beginners can learn it quickly.

2. Easy to Read:

Python code is easy to read and understand, which makes programming simpler.

3. Less Code:

Python programs can be written with fewer lines of code compared to many other languages.

4. Free to Use:

Python is free to download and use by anyone.

5. Platform Independent:

Python programs can run on different operating systems like Windows, Linux, and macOS.

6. Large Library Support:

Python provides many built-in libraries that help programmers perform tasks easily.

7. Used in Many Fields:

Python is used for web development, artificial intelligence, data science, automation, and game development.

8. Easy Debugging:

Since Python is an interpreted language, it is easier to find and fix errors.

Q:4) Installing Python and setting up the development environment (Anaconda, PyCharm, or VS Code).

Ans: Installing Python:

1. **Visit the official Python website: Python.org**
2. **Download the latest version of Python.**
3. **Run the installer.**
4. **Check the option "Add Python to PATH".**
5. **Click Install Now and wait for the installation to finish.**

Q:5) Writing and executing your first Python program.

Ans:

1. Open Your Python Editor:

Open python editor like vs code

2. Create a New Python File:

Create a new file and save it with the .py extension.

3.Write the Python Code:

Write a code like: print(“hello, world”)

4.Save the File:

Save the program after writing the code.

5. Run the Program:

Click the Run button or execute the file from the terminal.

2. Programming Style:

Q:1) Understanding Python's PEP 8 guidelines.

Ans: PEP 8 stands for **Python Enhancement Proposal 8**. It is a set of coding style guidelines that helps programmers write clean, readable, and consistent Python code.

Following PEP 8 makes code easier to understand and maintain, especially when working with other programmers.

- Basic PEP 8 Guidelines:
  1. Use Proper Indentation
  2. Use Meaningful Variable Names
  3. Add Spaces Around Operators
  4. Use Lowercase Names for Variables
  5. Write Comments When Needed

Q:2) Indentation, comments, and naming conventions in Python.

Ans: Python follows certain rules and conventions that make programs easy to read and understand. Three important concepts are **indentation**, **comments**, and **naming conventions**.

1) Indentation:
   Indentation means leaving spaces at the beginning of a line of code. In Python, indentation is very important because it defines blocks of code.

2) Comments:
   Comments are used to explain the code. They are ignored by Python during execution.

3) Naming Conventions:
   Naming conventions are rules for giving names to variables, functions, and other identifiers.

Q:3) Writing readable and maintainable code.

Ans: Readable and maintainable code is code that is easy to understand, modify, and debug. Good coding practices help programmers and others understand the program easily.

1. Use Meaningful Variable Names

Choose names that describe the purpose of the variable.

2. Use Proper Indentation

Python recommends using 4 spaces for indentation.

3. Add Comments

Comments help explain the code.

4. Keep Code Simple

Write simple and easy-to-understand statements.

3. Core Python Concepts:

Q:1) Understanding data types: integers, floats, strings, lists, tuples, dictionaries, sets.

Ans: A **data type** defines the type of value stored in a variable. Python provides different data types to store numbers, text, and collections of data.

1. Integer (int)

Integers are whole numbers without decimal points.

2. Float (float)

Floats are numbers with decimal points.

3. String (str)

Strings are used to store text and are written inside quotes.

4. List (list)

A list stores multiple values in a single variable. Lists are enclosed in square brackets [].

5. Tuple (tuple)

A tuple is similar to a list, but its values cannot be changed. Tuples are enclosed in parentheses ().

6. Dictionary (dict)

A dictionary stores data in **key-value pairs** and is enclosed in curly braces {}.

7. Set (set)

A set stores unique values and does not allow duplicate items.

Q:2) Python variables and memory allocation.

Ans: A **variable** is a name used to store data in a program. Variables help us save and use values whenever needed.

- **What is a Variable?**
  A variable stores information such as numbers, text, or other data types.

  For example:
  name = "Rahul"
  age = 18

  print(name)
  print(age)

  here, name stores the value “Rahul”.
  Age stores the value “18”.

- Memory Allocation in Python
  When a variable is created, Python automatically allocates memory to store its value.

For example:

X = 10

Here, Python creates the value 10 in memory. The variable x refers to that memory location. Whenever x is used, Python retrieves the value from memory.

Q:3) Python operators: arithmetic, comparison, logical, bitwise.

Ans: Operators are special symbols used to perform operations on variables and values. Python provides different types of operators such as arithmetic, comparison, logical, and bitwise operators.

1. Arithmetic Operators:

Arithmetic operators are used to perform mathematical calculations.

1) +
2) –
3) *
4) /
5) %
6) **

2. Comparison Operators:

Comparison operators compare two values and return either **True** or **False**.

1) ==
2) !=
3) >
4) <
5) >=
6) <=

3. Logical Operators

Logical operators are used to combine conditions.

1) And
2) Or
3) Not

4. Conditional Statements:

Q:1) Introduction to conditionalstatements: if, else, elif.

Ans: Conditional statements are used to make decisions in a Python program. They allow the program to execute different blocks of code based on whether a condition is **True** or **False**.

1. if Statement

The if statement executes a block of code only when the condition is true.

For example:

```
age = 18


if age >= 18:
    print("You are eligible to vote.")
```

2. else Statement

The else statement executes a block of code when the condition in the if statement is false.

For example:

```
age = 15


if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

3. elif Statement:

The elif (else if) statement is used to check multiple conditions.

For example:

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

Q:2) Nested if-else conditions.

Ans: A **nested if-else** condition means using an if or else statement inside another if or else statement. It is used when we need to check multiple conditions one after another.

For example:

```
age = 20
if age >= 18:
    if age >= 21:
        print("You can vote and drive.")
    else:
        print("You can vote.")
else:
    print("You are not eligible to vote.")
```

5. Looping (For, While):

Q:1) Introduction to for and while loops.

Ans: Loops are used in Python to execute a block of code repeatedly. Instead of writing the same code multiple times, we can use loops to perform repetitive tasks easily.

Python mainly provides two types of loops:

1. **for loop**
2. **while loop**

1. For Loop

A **for loop** is used when we know how many times we want to repeat a task.

- range(5) generates numbers from 0 to 4.
- The loop runs 5 times.
- Each time, it prints "Hello".

2. While Loop

A **while loop** is used when we want to repeat a task as long as a condition is true.

- The loop starts with i = 1.

- It continues while i <= 5.
- After each iteration, i is increased by 1.

Q:2) How loops work in Python.

Ans: Loops are used in Python to repeat a block of code multiple times. Instead of writing the same code again and again, we can use loops to perform repetitive tasks easily.

How a Loop Works

A loop follows these steps:

1. Starts the loop.
2. Checks a condition or range.
3. Executes the code inside the loop.
4. Repeats the process until the condition becomes false or the range ends.
5. Exits the loop.

Q:3) Using loops with collections (lists, tuples, etc.).

Ans: In Python, collections such as **lists**, **tuples**, **sets**, and **dictionaries** can store multiple values. Loops are often used to access and display each item in these collections one by one.

- **Using a Loop with a List**

  A list stores multiple items in square brackets [].

- **Using a Loop with a List**

  A list stores multiple items in square brackets [].

- **Using a Loop with a Set**

  A set stores unique values in curly braces {}.

- **Using a Loop with a Dictionary**

  A dictionary stores data in key-value pairs.

6. Generators and Iterators:

Q:1) Understanding how generators work in Python.

Ans: A **generator** is a special type of function in Python that produces values one at a time instead of returning all values at once. Generators help save memory and are useful when working with large amounts of data.

Generators use the **yield** keyword instead of the **return** keyword. Generators use less memory and generate value only when needed it is also Useful for handling large data-sets.

Q:2) Difference between yield and return.

Ans: return is used when a function needs to send back a value and then stop. yield is used in generator functions to produce values one by one without ending the function immediately. For beginners, return is used more often, while yield is useful when working with large amounts of data.

Q:3) Understanding iterators and creating custom iterators.

Ans: An **iterator** is an object that allows us to access elements of a collection one by one. Python uses iterators in loops such as for loops.

Lists, tuples, strings, and other collections are iterable objects.

An iterator returns one item at a time from a collection.

- We can create our own iterator using a class. __iter__() returns the iterator object.
- __next__() returns the next value.
- StopIteration stops the loop when all values have been returned.

7. Functions and Methods:

Q:1) Defining and calling functions in Python.

Ans: A **function** is a block of code that performs a specific task. Functions help us avoid writing the same code multiple times and make programs easier to understand.

- **Defining a Function:**

  In Python, a function is defined using the def keyword.

- **Calling a Function:**

  To use a function, we need to call it by its name.

Functions are an important part of Python programming. They allow programmers to group code into reusable blocks. A function is created using the def keyword and executed by calling its name. Functions make programs shorter, cleaner, and easier to maintain.

Q:2) Function arguments (positional, keyword, default).

Ans: Function arguments are values passed to a function when it is called. They allow a function to work with different data.

1. Positional Arguments

In positional arguments, values are passed in the same order as the parameters in the function definition.

2. Keyword Arguments

In keyword arguments, values are passed using parameter names.

3. Default Arguments

Default arguments have a predefined value. If no value is provided, the default value is used.

In conclusion, Function arguments help pass data to functions. **Positional arguments** depend on the order of values, **keyword arguments** use parameter names, and **default arguments** provide a predefined value when no argument is supplied. These features make Python functions more useful and flexible.

Q:3) Scope of variables in Python.

Ans: The **scope of a variable** refers to the area of a program where a variable can be accessed or used.

In Python, variables mainly have two types of scope:

1. **Local Scope**
2. **Global Scope**

1. Local Scope

A variable created inside a function is called a **local variable**. It can only be used inside that function.

2. Global Scope

A variable created outside all functions is called a **global variable**. It can be used anywhere in the program.

Variable scope determines where a variable can be used in a program. **Local variables** are created inside functions and can only be used there, while **global variables** are created outside functions and

can be used throughout the program. Understanding variable scope helps beginners write better and more organized Python code.

Q:4) Built-in methods for strings, lists, etc.

Ans: Python provides many **built-in methods** that make it easy to work with strings, lists, and other data types. These methods help perform common tasks such as changing text, adding items, and sorting data.

1. String Methods

String methods are used to manipulate text.

2. List Methods

List methods are used to add, remove, and modify items in a list.

3. Tuple Methods

Tuples have fewer methods because they cannot be changed.

4. Dictionary Methods

Dictionary methods help manage key-value pairs.

8. Control Statements (Break, Continue, Pass):

Q:1) Understanding the role of break, continue, and pass in Python loops.

Ans: In Python, **break**, **continue**, and **pass** are special statements used to control the flow of loops. They help programmers decide how a loop should behave in different situations.

1. break Statement

The break statement is used to stop a loop immediately.

2. continue Statement

The continue statement skips the current iteration and moves to the next iteration of the loop.

3. pass Statement

The pass statement does nothing. It is used as a placeholder when a statement is required but no code is written yet

9. String Manipulation:

Q:1) Understanding how to access and manipulate strings.

Ans: A **string** is a sequence of characters used to store text. In Python, strings are written inside single quotes (' ') or double quotes (" ").

1. Accessing Characters in a String

Each character in a string has an index number. Indexing starts from 0.

2. Accessing Multiple Characters (Slicing)

Slicing is used to get a part of a string.

3. Converting Case

Python provides methods to change the case of a string.

4. Replacing Text

The replace() method replaces one word with another.

5. Finding the Length of a String

The len() function returns the number of characters in a string.

6. Joining Strings

Strings can be combined using the + operator.

Q:2) Basic operations: concatenation, repetition, string methods (upper(), lower(), etc.).

Ans: Strings are used to store text in Python. Python provides several operations and methods to work with strings easily, such as **concatenation**, **repetition**, and **string methods**.

1. String Concatenation

Concatenation means joining two or more strings using the + operator.

2. String Repetition

Repetition means repeating a string multiple times using the * operator.

3. String Methods

Python provides built-in methods to manipulate strings.

a) upper()

Converts all characters to uppercase.

b) lower()

Converts all characters to lowercase.

c) title()

Capitalizes the first letter of each word.

d) replace()

Replaces one word with another.

e) strip()

Removes extra spaces from the beginning and end of a string.

Q:3) String slicing.

Ans: **String slicing** is used to extract a part of a string. It allows us to access a group of characters from a string instead of a single character.

For example:

name = "Python"

print(name[0:3])

Here, 0 is starting point and 3 is an ending point.

10. Advanced Python (map(), reduce(), filter(), Closures and Decorators):

Q:1) How functional programming works in Python

Ans: **Functional programming** is a programming style where functions are used to perform tasks and process data. Instead of changing data repeatedly, functions take input, process it, and return output.

Python supports functional programming along with other programming styles.

Main Features of Functional Programming

1. Functions are Treated as Objects

In Python, functions can be stored in variables and passed to other functions.

2. Using map()

The map() function applies a function to each item in a list.

3. Using filter()

The filter() function selects items that satisfy a condition.

4. Using lambda Functions

A lambda function is a small anonymous function written in a single line.

Q:2) Using map(), reduce(), and filter() functions for processing data.

Ans: Python provides built-in functions such as **map()**, **filter()**, and **reduce()** to process data easily. These functions help perform operations on lists and other collections.

1. map() Function

The map() function applies a function to each item in a collection and returns the modified values.

2. filter() Function

The filter() function selects only those items that satisfy a condition.

3. reduce() Function

The reduce() function combines all items in a collection into a single value. It is available in the functools module.

Q:3) Introduction to closures and decorators.

Ans: Closures and decorators are advanced features in Python, but beginners can understand their basic idea easily.

A **closure** is a function inside another function. The inner function can remember and use variables from the outer function even after the outer function has finished.

A **decorator** is a function that adds extra functionality to another function without changing its original code.

A **closure** is an inner function that remembers variables from its outer function. A **decorator** is a special function that adds extra behavior to another function. These concepts are powerful features of Python and are commonly used in advanced Python programming.